

# Chapter 48. Big Data & Distributed Compute

Most big data isn't: partitions, workers and shuffles, PySpark on 20 million sensor readings, reading query plans, pruning and broadcast joins, and a timed comparison with DuckDB and Polars.

Draft chapter

18 September 2026

Every output from a real run

# Contents

## Chapter 48. Big Data & Distributed Compute

Why this matters

In plain English

48.1 Most "big data" isn't

48.2 What "distributed" actually means

48.3 PySpark on the sensor data

48.4 Reading the plan, and finding the shuffle

48.5 Making a job faster

48.6 The same question in three tools

48.7 Writing Spark that doesn't surprise you

48.8 The wider landscape in one page

Common mistakes and how to spot them

In the real world: the cluster that was slower than the laptop

Tools

The project: three questions, three engines

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

# Chapter 48. Big Data & Distributed Compute

Part V — Data Engineering, Integration & Scale

## Chapter at a glance

**You will learn to:** judge for yourself whether a dataset needs distributed computing at all · explain partitions, workers, the driver, and why moving data between machines is what costs · write PySpark that reads, filters, groups, and joins Riverstone's sensor data · read a physical query plan and find the shuffle in it · use partition pruning, column pruning, and broadcast joins, and see each one change the plan · recognize skew and know the usual fixes · compare DuckDB, Polars, and Spark on the same question and interpret the timings · avoid the common ways a Spark job surprises you · place Databricks, EMR, Dataproc, Fabric, Trino, and Ray on the map.

**Before you start:** Chapter 18 (Python for analysis), Chapter 29 (Python as software), Chapter 45 (files and Parquet), and Chapter 46 (pipelines). Chapter 49 covers the storage side in depth.

**Time needed:** 12–16 hours of reading and practice, spread over two to three weeks.

**Tools:** Python 3.12 with `pyspark`, `duckdb`, `polars`, and `pyarrow`, plus a Java runtime (Java 17 or 21) for Spark. Everything runs on one machine.

**Practice data:** Riverstone's plant sensor readings, generated by the companion script: 25 machines across two plants, one reading every 10 seconds from 1 October to 31 December 2025. That's **19,872,000 readings**, about 114 MB as Parquet.

---

## Why this matters

Somewhere in every data engineer's first year, a dataset stops fitting comfortably in memory, a job that took two minutes starts taking two hours, and somebody says the word "Spark".

Two mistakes follow. The first is reaching for a cluster far too early, which is expensive, slow to develop on, and harder to debug than the problem it replaced. The second is refusing to learn distributed computing at all, and then writing single-machine code that quietly can't grow, or Spark code that works but costs ten times what it should because of one line.

This chapter gives you the judgment and the mechanics. You'll write real PySpark on 20 million sensor readings, read the plans Spark makes, watch a shuffle appear and then make it disappear, and time the same question in three tools. By the end you should be able to answer, with reasons and numbers, the question every interviewer eventually asks: *"How would you process this, and why not something simpler?"*

---

# In plain English

Think about counting the votes in an election.

- One counting officer can handle a small booth. That's a **single machine**, and it's usually enough.
- For a whole state, you divide ballots into boxes and hand them to many officers. Each box is a **partition**; each officer is a **worker**.
- A supervisor tells everyone what to count and collects the totals at the end. That's the **driver**.
- Counting inside one box is fast. The expensive part is when ballots must be **moved between officers**, for example to put all the votes from one constituency together. That's a **shuffle**, and it's the single most important idea in this chapter.
- If every officer gets a small box except one who gets a truckload, everyone waits for that one person. That's **skew**.
- If you only need votes from one district, you shouldn't open every box. That's **partition pruning**.
- If you only need the party name and not the whole ballot, you shouldn't read the whole ballot. That's **column pruning**.
- Hiring a thousand officers to count a village's votes is slower than one person doing it, because organizing them takes longer than the counting. That's Spark on small data.

## 48.1 Most "big data" isn't

The draft of this book made a claim worth repeating, because it has only become more true: **most datasets described as "big data" fit on one computer**.

A modern laptop has 8 to 32 GB of memory and 8 or more cores. Tools such as DuckDB and Polars are built to use them, to work directly on Parquet files, and to spill to disk when data doesn't fit in memory. In practice that means:

| Rough size                       | What usually works                                                                      |
|----------------------------------|-----------------------------------------------------------------------------------------|
| Up to a few million rows         | Spreadsheets struggle; SQL in a database, pandas, DuckDB, or Polars are all comfortable |
| Tens of millions of rows         | DuckDB or Polars on a laptop; a warehouse if it's shared                                |
| Hundreds of millions to billions | A cloud warehouse (Chapter 49), or Spark, usually on a cluster                          |
| Continuous, never-ending data    | Streaming (Chapter 50), which is a different problem                                    |

Riverstone's entire order history is a few hundred thousand rows: a spreadsheet's worth. Its sensor data is far bigger, and this chapter's 19.87 million readings are three months from 25 machines. A year across more machines reaches hundreds of millions, which is where the question becomes real.

Before choosing a distributed tool, ask four questions.

1. **How much data does the job actually touch?** Not the size of the source; the size after filtering to the days and columns you need. Partitioned Parquet often turns "a billion rows" into "40 million".
2. **How often does it run, and how long can it take?** A nightly job with a four-hour window is a different problem from a query someone waits for.
3. **Does it fit in memory after the filter?** If yes, one machine is almost always faster and simpler.
4. **What's the cost of being wrong?** Starting simple and moving to Spark later is cheap. Starting on a cluster and discovering you never needed it costs months of complexity.

**Watch out: "we might need it later" is not a reason.** Later is when to build it. A pipeline written in SQL or Polars can be rewritten in Spark in days, and most never need to be.

---

## 48.2 What "distributed" actually means

---

A distributed engine splits data into **partitions**, sends work on each partition to a **worker** (also called an executor), and coordinates everything from the **driver**, which is the process running your code.

Three consequences shape everything you write.

- **Work that stays inside a partition is cheap.** Filtering, selecting columns, arithmetic, and per-row work run in parallel with no coordination. These are **narrow** operations.
- **Work that needs data from other partitions is expensive.** Grouping, joining, sorting, and deduplication must first bring matching rows together, which means writing data to disk, sending it over the network, and reading it back. That movement is the **shuffle**, and these are **wide** operations.
- **Bringing results back to the driver is a bottleneck.** `collect()` pulls every row into the memory of one process. It's how many first Spark jobs die.

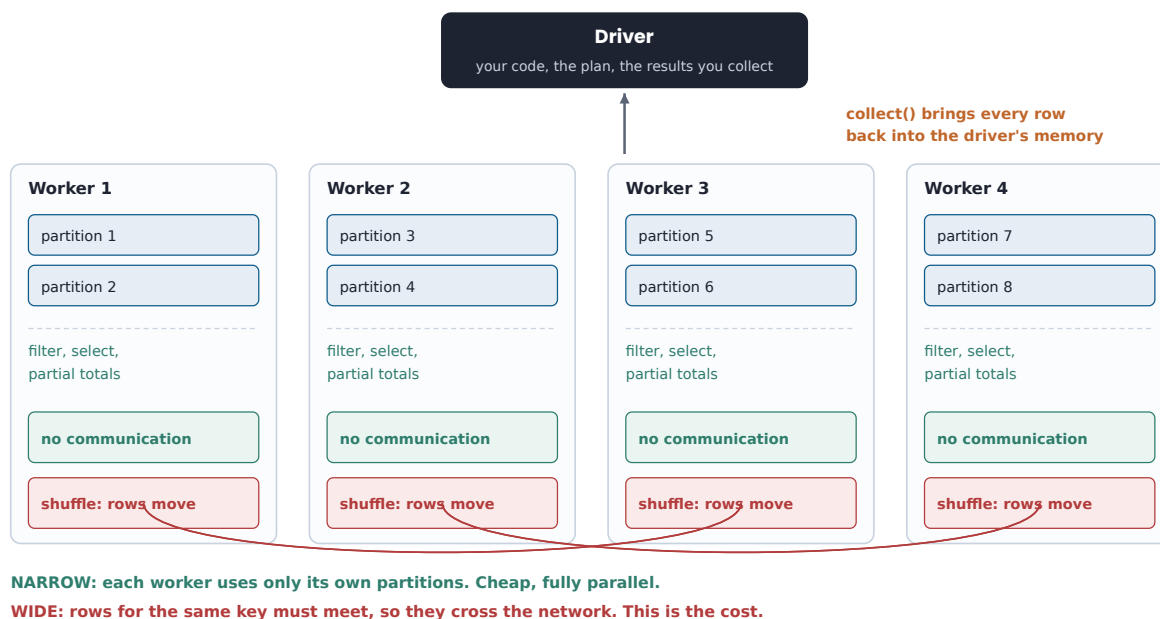


Figure 48.1 – Driver, workers, partitions, and the shuffle. The crossing arrows are what you pay for.

**Lazy execution** ties it together. Spark doesn't run anything as you build a query. It records what you asked for, plans it as a whole, and runs it when you ask for a result (an **action** such as `count()`, `show()`, `collect()`, or writing files). That's why Spark can rearrange your work, and why the plan is worth reading.

**Simplification note.** Everything here runs Spark in **local mode**, one JVM on one machine, using your cores as workers. That's the standard way to develop and test, and every idea, plan, and optimization transfers to a cluster. What local mode can't show you is network cost and cluster tuning; section 48.6 is honest about what that means for the timings.

## 48.3 PySpark on the sensor data

The companion script generates the dataset once:

```
python3 make_sensor_data.py          # 19,872,000 readings, ~114 MB, about 2-4 minutes
python3 make_sensor_data.py --small  # one day, 216,000 readings, for quick experiments
```

It writes Parquet files partitioned by `reading_date`, one folder per day, which is how sensor and event data is usually stored (Chapter 49 explains why).

```

import os, time
import duckdb, polars as pl
from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder
        .appName("riverstone-sensors")
        .master("local[*]")                # every core on this machine, no cluster
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.ui.showConsoleProgress", "false")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

import re

def plan(df, only=None):
    """Print a physical plan with the ids that change between runs taken out."""
    text = df._jdf.queryExecution().executedPlan().toString()
    text = re.sub(r"#\d+L?", "", text)      # attribute ids
    text = re.sub(r", \[plan_id=\d+\]", "", text) # plan ids
    text = re.sub(r"InMemoryFileIndex\(\d+ paths\)\[^\]*\\", "sensor_readings", text)
    out = []
    for line in text.splitlines():
        if "FileScan" in line:              # split the long scan line into its
parts
            head, *parts = line.split(", ")
            out.append(head.rstrip())
            out += [" " + p for p in parts
                    if p.startswith(("PartitionFilters", "PushedFilters", "ReadSchema"))]
        else:
            out.append(line.rstrip()[:96])
    print("\n".join(l for l in out if not only or any(k in l for k in only)))

readings = spark.read.parquet("sensor_readings")
print("rows:", f"{readings.count():,}")
readings.printSchema()

```

```

rows: 19,872,000
root
|-- machine_id: string (nullable = true)
|-- plant: string (nullable = true)
|-- line_type: string (nullable = true)
|-- reading_ts: timestamp_ntz (nullable = true)
|-- temperature_c: double (nullable = true)
|-- pressure_bar: double (nullable = true)
|-- units_made: integer (nullable = true)
|-- scrap_flag: boolean (nullable = true)
|-- reading_date: date (nullable = true)

```

### How it works, line by line.

- `SparkSession.builder` creates the entry point. `master("local[*]")` means "run locally, using every core on this machine"; on a cluster this is where the cluster manager goes instead.
- `spark.sql.shuffle.partitions` sets how many pieces the data is cut into after a shuffle. The default is 200, which is far too many for one machine and a common cause of slow local jobs. Eight is sensible here; on a cluster, aim for a few times the number of cores.
- `spark.read.parquet()` reads the folder. Spark discovers the `reading_date` partition column from the folder names, which is why it appears in the schema without being in any file.

- The `plan()` helper prints a physical plan with the ids that change between runs removed, so the plans in this book match what you'll see.

## A first real question

"What's the average temperature and scrap rate for each plant and line type?"

```
by_plant = (readings
    .groupBy("plant", "line_type")
    .agg(F.count("*").alias("readings"),
        F.round(F.avg("temperature_c"), 2).alias("avg_temp_c"),
        F.round(F.avg(F.col("scrap_flag").cast("int")) * 100, 2).alias("scrap_pct"))
    .orderBy("plant", "line_type"))
by_plant.show(truncate=False)
```

```
+-----+-----+-----+-----+
|plant      |line_type|readings|avg_temp_c|scrap_pct|
+-----+-----+-----+-----+
|Bhiwandi Main|Blow      |3974400 |196.34    |1.3       |
|Bhiwandi Main|Extrusion |3974400 |196.35    |1.29      |
|Bhiwandi Main|Injection |3974400 |196.35    |1.29      |
|Chakan Pune  |Blow      |2384640 |196.35    |1.3       |
|Chakan Pune  |Extrusion |3179520 |196.32    |1.3       |
|Chakan Pune  |Injection |2384640 |196.38    |1.29      |
+-----+-----+-----+-----+
```

**Reading it.** Three line types at each plant, with scrap around 1.3% and average temperature near 196 °C. Bhiwandi Main has more readings because it has more machines (15 of the 25). This is a **wide** operation: the group-by needed a shuffle, which section 48.4 makes visible.

The PySpark DataFrame API will feel familiar after pandas and Polars: `select`, `filter`, `groupBy`, `agg`, `join`, `orderBy`, `withColumn`. Spark also accepts SQL directly (`readings.createOrReplaceTempView("readings"); spark.sql("SELECT ...")`), and the two are the same engine underneath. Use whichever makes the logic clearer.

## Lazy execution, seen

```
hot = readings.filter(F.col("temperature_c") > 200).select("machine_id", "reading_ts",
    "temperature_c")
print("nothing has run yet:", type(hot).__name__)

start = time.perf_counter()
hot_count = hot.count() # this is the action that runs the job
print(f"readings above 200 C: {hot_count:,}")
```

```
nothing has run yet: DataFrame
readings above 200 C: 8,143,256
```

**Reading it.** Building `hot` was instant: no data was touched. `count()` is an action, so Spark planned and ran the job, scanning 19.87 million readings to find 8.14 million above 200 °C. On a cluster, the same code would run the same way across machines.

## 48.4 Reading the plan, and finding the shuffle

A **query plan** is Spark's description of how it will do the work. Reading it is the difference between guessing at performance and knowing.

```
plan(readings.groupBy("machine_id").agg(F.sum("units_made").alias("units")))
```

```
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[machine_id], functions=[sum(units_made)], output=[machine_id, units])
  +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS
    +- HashAggregate(keys=[machine_id], functions=[partial_sum(units_made)], output=[machine_id, units_made])
      +- Project [machine_id, units_made]
        +- FileScan parquet [machine_id,units_made,reading_date] Batched: true
          PartitionFilters: []
          PushedFilters: []
          ReadSchema: struct<machine_id:string,units_made:int>
```

**How to read it**, from the bottom up, because that's the order things happen.

- **FileScan parquet** reads the files. **ReadSchema** shows Spark only reads two columns, **machine\_id** and **units\_made**. That's **column pruning**, free with Parquet.
- **Project** keeps only the columns needed.
- **HashAggregate ... partial\_sum** is a partial total computed inside each partition, with no communication. Doing this before the shuffle is why grouping is efficient: far less data has to move.
- **Exchange hashpartitioning(machine\_id, 8)** is the **shuffle**: rows are redistributed so that every row with the same **machine\_id** ends up on the same worker. **ENSURE\_REQUIREMENTS** means Spark added it because the next step needs it.
- **HashAggregate** at the top combines the partial totals into final ones.
- **AdaptiveSparkPlan** means adaptive query execution is on: Spark may change parts of the plan at runtime once it sees the real data sizes.

**Every Exchange is a shuffle, and every shuffle costs.** When a job is slow, count the exchanges first.

### Group by machine: one shuffle

```
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[machine_id], sum(units_made))
  +- Exchange hashpartitioning(machine_id, 8)
    +- HashAggregate(partial_sum(units_made))
      +- Project [machine_id, units_made]
        +- FileScan parquet [machine_id, units_made]
```

**Exchange = the shuffle: rows for a machine move together**

### One day, two columns: no shuffle

```
*(1) Project [machine_id, temperature_c]
+- *(1) Filter (plant = Bhiwandi Main)
  +- *(1) ColumnarToRow
    +- FileScan parquet
```

**PartitionFilters:** [reading\_date = 2025-12-01]

**PushedFilters:** [EqualTo(plant, Bhiwandi Main)]

**ReadSchema:** machine\_id, plant, temperature\_c

**PartitionFilters and ReadSchema = work never done**

**Read a plan from the bottom up. Count the exchanges: each one writes data to disk, sends it, and reads it back.**

Then look at the scan: **PartitionFilters** skip whole folders, **ReadSchema** shows which columns are read at all.

Figure 48.2 – The two things to look for in a plan: exchanges (cost) and pruning (work avoided).

## 48.5 Making a job faster

### Partition pruning: don't open boxes you don't need

```
december = readings.filter((F.col("reading_date") == "2025-12-01") & (F.col("plant") == "Bhiwandi Main"))
print("rows for 1 December at Bhiwandi Main:", f"{december.count():,}")
plan(december.select("machine_id", "temperature_c"))
```

```
rows for 1 December at Bhiwandi Main: 129,600
*(1) Project [machine_id, temperature_c]
+- *(1) Filter (isnotnull(plant) AND (plant = Bhiwandi Main))
   +- *(1) ColumnarToRow
      +- FileScan parquet [machine_id,plant,temperature_c,reading_date] Batched: true
         PartitionFilters: [isnotnull(reading_date)]
         PushedFilters: [IsNotNull(plant)]
         ReadSchema: struct<machine_id:string,plant:string,temperature_c:double>
```

**Reading it.** The plan has **no Exchange at all** (this query needs no shuffle), and two lines matter:

- **PartitionFilters:** `[... reading_date = 2025-12-01]` means Spark reads one day's folder out of 92. It never opens the other 91.
- **PushedFilters:** `[... EqualTo(plant, Bhiwandi Main)]` means the plant filter is handled while reading the Parquet file, using the file's own statistics, instead of reading everything and filtering afterwards.

Between them, a query over 19.87 million readings touched 216,000. **Partitioning your storage by the column people filter on** (usually a date) is the single biggest performance decision in a data platform.

**Watch out: a filter Spark can't see doesn't prune.** Filtering with a Python function (a user-defined function), or on a column derived from the partition column (`F.year(F.col("reading_ts")) == 2025`), hides the filter from the planner, and Spark reads everything. Filter on the partition column directly, and check `PartitionFilters` in the plan.

### Joins: the shuffle you can often avoid

Joining the readings to a small table of machine supervisors needs matching rows together, which normally means shuffling **both** sides:

```
machines = spark.createDataFrame(
    [("M-01", "Anil Deshmukh", 2019), ("M-02", "Sunita Rao", 2021), ("M-03", "Imran Shaikh", 2017)],
    ["machine_id", "supervisor", "installed_year"])

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1) # pretend Spark can't tell the table is small
joined = readings.join(machines, on="machine_id",
    how="inner").groupBy("supervisor").agg(F.count("*").alias("readings"))
plan(joined, only=["Join", "Exchange"])
print()
print("supervisor totals:")
joined.orderBy("supervisor").show(truncate=False)
```

```

+- Exchange hashpartitioning(supervisor, 8), ENSURE_REQUIREMENTS
  +- SortMergeJoin [machine_id], [machine_id], Inner
    : +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS
      +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS

supervisor totals:
+-----+-----+
|supervisor|readings|
+-----+-----+
|Anil Deshmukh|794880|
|Imran Shaikh|794880|
|Sunita Rao|794880|
+-----+-----+

```

**Reading it.** `SortMergeJoin` with two exchanges below it: both sides are redistributed by `machine_id`, and 19.87 million rows cross the network to be matched with three rows. The results are right, and the work is wasteful.

The fix is a **broadcast join**: send the small table to every worker, so the big table never moves.

```

broadcast_join = (readings.join(F.broadcast(machines), on="machine_id")
                  .groupBy("supervisor").agg(F.count("*").alias("readings")))
plan(broadcast_join, only=["Join", "Exchange"])
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10 * 1024 * 1024) # back to the default

```

```

+- Exchange hashpartitioning(supervisor, 8), ENSURE_REQUIREMENTS
  +- BroadcastHashJoin [machine_id], [machine_id], Inner, BuildRight, false, false
    +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]), false)

```

**Reading it.** `BroadcastHashJoin` with a `BroadcastExchange` for the small side, and **no exchange on the big side**. The big data stays where it is. Spark does this automatically when it can tell a table is small (`spark.sql.autoBroadcastJoinThreshold`, 10 MB by default); `F.broadcast()` tells it explicitly, which matters when the size estimate is wrong, for example after a complex sub-query.

The remaining `Exchange hashpartitioning(supervisor, 8)` is the group-by, which still needs a shuffle. One shuffle instead of three.

## Skew: when one worker gets a truckload

**Skew** is uneven partitions. If half the readings come from one machine, the worker holding that machine's rows does half the work while others idle. Symptoms: a stage where 199 tasks finish in seconds and one runs for an hour, and memory errors on a single task.

Usual fixes:

- **Adaptive query execution** (on by default in Spark 3 and later) detects skewed join partitions and splits them. Often this is enough.
- **Salting**: add a random number to the join key on the big side and expand the small side to match, spreading the hot key across workers.
- **Handle the hot key separately**: filter it out, process it on its own, and union the results.
- **Filter earlier**, so the skewed rows never reach the join.

```
# Salting, in outline: spread one hot machine across 8 partitions
from pyspark.sql import functions as F

SALTS = 8
big = readings.withColumn("salt", (F.rand() * SALTS).cast("int"))
small = machines.withColumn("salt", F.explode(F.array([F.lit(i) for i in range(SALTS)])))
joined = big.join(small, on=["machine_id", "salt"]) # the hot key is now 8 keys
```

## A short list of what actually helps

| Change                                       | Why it helps                       | Look for in the plan                            |
|----------------------------------------------|------------------------------------|-------------------------------------------------|
| Partition storage by date                    | Whole folders are skipped          | PartitionFilters                                |
| Select only needed columns                   | Less data read and moved           | ReadSchema                                      |
| Filter before joining and grouping           | Less data shuffled                 | Filters below the Exchange                      |
| Broadcast small tables                       | The big side never moves           | BroadcastHashJoin , no Exchange on the big side |
| Set <code>shuffle.partitions</code> sensibly | Avoids thousands of tiny tasks     | Number in <code>hashpartitioning(..., n)</code> |
| Write files of a sensible size (128-512 MB)  | Fewer, bigger reads                | Number of output files                          |
| Cache only what's reused several times       | Avoids recomputation; costs memory | InMemoryTableScan                               |

## 48.6 The same question in three tools

Now the comparison the blueprint asks for: one question, three engines.

*"For readings above 190 °C, what's the average temperature, scrap rate, and total output per plant?"*

```

QUESTION = """
    SELECT plant,
           ROUND(AVG(temperature_c), 2)           AS avg_temp_c,
           ROUND(AVG(CAST(scrap_flag AS INT)) * 100, 2) AS scrap_pct,
           SUM(units_made)::BIGINT                AS units
    FROM readings
    WHERE temperature_c > 190
    GROUP BY plant
    ORDER BY plant"""

con = duckdb.connect()
con.execute("CREATE VIEW readings AS SELECT * FROM read_parquet('sensor_readings/**/*.parquet')")
print("DuckDB:")
print(con.execute(QUESTION).df().to_string(index=False))

lf = pl.scan_parquet("sensor_readings/**/*.parquet")
polars_answer = (lf.filter(pl.col("temperature_c") > 190)
                .group_by("plant")
                .agg([(pl.col("temperature_c").mean().round(2)).alias("avg_temp_c"),
                     (pl.col("scrap_flag").cast(pl.Int8).mean() *
                      100).round(2).alias("scrap_pct"),
                     pl.col("units_made").sum().alias("units")])
                .sort("plant").collect())

print("\nPolars:")
print(polars_answer)

print("\nSpark:")
(readings.filter(F.col("temperature_c") > 190)
 .groupBy("plant")
 .agg(F.round(F.avg("temperature_c"), 2).alias("avg_temp_c"),
      F.round(F.avg(F.col("scrap_flag").cast("int")) * 100, 2).alias("scrap_pct"),
      F.sum("units_made").alias("units"))
 .orderBy("plant").show(truncate=False))

```

DuckDB:

| plant         | avg_temp_c | scrap_pct | units    |
|---------------|------------|-----------|----------|
| Bhiwandi Main | 203.43     | 1.33      | 23234149 |
| Chakan Pune   | 203.44     | 1.33      | 15491402 |

Polars:

shape: (2, 4)

| plant         | avg_temp_c | scrap_pct | units    |
|---------------|------------|-----------|----------|
| ---           | ---        | ---       | ---      |
| str           | f64        | f64       | i32      |
| Bhiwandi Main | 203.43     | 1.33      | 23234149 |
| Chakan Pune   | 203.44     | 1.33      | 15491402 |

Spark:

```

+-----+-----+-----+-----+
|plant      |avg_temp_c|scrap_pct|units  |
+-----+-----+-----+-----+
|Bhiwandi Main|203.43    |1.33     |23234149|
|Chakan Pune  |203.44    |1.33     |15491402|
+-----+-----+-----+-----+

```

**Reading it.** All three agree to the last unit: 203.43 °C and 1.33% scrap at Bhiwandi Main, 23,234,149 units. That's the first thing to check when comparing tools, and it's reassuring how often people skip it.

## Timings

Speed depends on the machine, so these numbers are **measurements from one specific setup, not a benchmark**. They were taken on the container used to write this book: **1 CPU core, 3 GB of memory, Python 3.12.3, DuckDB 1.5.5, Polars 1.44.2, PySpark 4.2.0, Java 21**. Each timing is the fastest of three runs, after the files were in the operating system's cache.

| Data                          | DuckDB | Polars | Spark (local) |
|-------------------------------|--------|--------|---------------|
| 216,000 readings (one day)    | 0.01 s | 0.00 s | 0.73 s        |
| 19,872,000 readings (92 days) | 0.38 s | 0.47 s | 2.01 s        |

### What this does and doesn't show.

- On a single machine, at this size, **DuckDB and Polars are several times faster than Spark**, and far simpler to run. On the small file, Spark's overhead dominates completely.
- The gap is not a flaw in Spark. Spark is paying for machinery that lets the same code run over hundreds of machines, and this machine has **one core**, so Spark's parallelism has nowhere to go.
- What these numbers *can't* show is the crossover: a dataset large enough that one machine runs out of memory or time, where Spark keeps going by adding machines. You'll reach that point with hundreds of millions to billions of rows, or with jobs that need more memory than a machine can hold.

**Try it.** Run the timing script in the companion folder on your own machine and compare. If you have 8 cores, you should see Spark's numbers improve and the small-file result stay bad. That difference is the whole argument of this section.

## Choosing

| Choose                       | When                                                                                                                                                   |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SQL in your warehouse</b> | The data is already there and the query is a query (most of the time)                                                                                  |
| <b>DuckDB</b>                | Files on disk or in cloud storage, SQL, one machine, up to tens of millions of rows and beyond                                                         |
| <b>Polars</b>                | The same, expressed as Python DataFrame code, or inside an existing Python pipeline                                                                    |
| <b>Spark</b>                 | Data too large for one machine, jobs that must scale with the business, streaming at volume, or a platform (Databricks, EMR) your company already runs |
| <b>A cluster of any kind</b> | Only when a single machine has been tried and can't do it                                                                                              |

## 48.7 Writing Spark that doesn't surprise you

- **Don't `collect()` big results.** It pulls every row into the driver's memory.

```
small = readings.filter((F.col("reading_date") == "2025-12-01") & (F.col("machine_id") == "M-07"))
rows = small.select("reading_ts", "temperature_c").limit(3).collect()
print("three rows brought back to Python:", [(str(r[0]), r[1]) for r in rows])
print("rows that would come back without the filter:", f"{readings.count():,}")
```

```
three rows brought back to Python: [('2025-12-01 00:00:00', 195.78), ('2025-12-01 00:00:10',
191.9), ('2025-12-01 00:00:20', 195.91)]
rows that would come back without the filter: 19,872,000
```

Bring back aggregates, samples, or filtered slices; write full results to files or a table instead.

- **Watch the number of output files.** Writing with 200 shuffle partitions gives 200 files. Thousands of small files make later reads slow (the "small files problem", Chapter 49). Use `repartition()` or `coalesce()` to aim for files of roughly 128–512 MB.
- **Cache deliberately.** `df.cache()` keeps a DataFrame in memory for reuse. It helps when the same intermediate result is used several times; otherwise it wastes memory you'd rather give to the shuffle.
- **Prefer built-in functions over Python UDFs.** A Python user-defined function moves every row out of the JVM into Python and back, and blocks the optimizer from pushing filters down. Built-ins in `pyspark.sql.functions` stay in the engine.
- **Test on a small partition.** Run your job on one day before running it on a year: same code, one folder.
- **Keep the driver small and the job idempotent.** Chapter 46's rules apply: writes replace partitions, and a re-run is safe.

## 48.8 The wider landscape in one page

| Thing                                                    | What it is                                                                            | Where you'll meet it                                |
|----------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>Apache Spark</b>                                      | The distributed engine in this chapter                                                | Almost everywhere; the default for large batch work |
| <b>Databricks</b>                                        | A managed platform built around Spark, with notebooks, Delta Lake, and job scheduling | Large enterprises; very common in job descriptions  |
| <b>Amazon EMR, Google Dataproc, Azure Synapse/Fabric</b> | Cloud-managed Spark (and friends)                                                     | Companies standardized on one cloud                 |
| <b>Delta Lake, Apache Iceberg, Apache Hudi</b>           | Table formats that add transactions and time travel to files                          | Chapter 49                                          |
| <b>Trino / Presto</b>                                    | A query engine that runs SQL across many sources without moving the data              | Ad-hoc analytics over a lake                        |
| <b>Ray, Dask</b>                                         | Distributed Python for machine learning and general workloads                         | Data science teams scaling Python                   |
| <b>Hadoop and MapReduce</b>                              | The original distributed platform; HDFS still appears                                 | Older systems; mostly historical now                |

| Thing                                | What it is                                                    | Where you'll meet it                 |
|--------------------------------------|---------------------------------------------------------------|--------------------------------------|
| <b>Snowflake, BigQuery, Redshift</b> | Cloud warehouses that scale SQL without you managing machines | Chapter 49; often the simpler answer |

The honest summary for a career: learn **SQL and one single-machine engine** deeply, learn **Spark's model** well enough to read plans and write correct jobs, and learn a **cloud platform** when a job needs one. Interviews for data engineering roles ask about Spark far more often than most day jobs use it, which is a reason to understand it, not a reason to build on it.

## Common mistakes and how to spot them

| Mistake                                                  | Symptom                                                 | Fix                                             |
|----------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------|
| Reaching for Spark on small data                         | A five-second job takes two minutes                     | Measure first; DuckDB or Polars (48.1, 48.6)    |
| Leaving <code>shuffle.partitions</code> at 200 locally   | Hundreds of tiny tasks, slow jobs                       | Set it to a few times your core count           |
| Not looking at the plan                                  | Guessing at why a job is slow                           | Read the plan; count the exchanges (48.4)       |
| Filtering on a derived column                            | <code>PartitionFilters</code> empty; whole dataset read | Filter on the partition column directly (48.5)  |
| Shuffling a big table to join a tiny one                 | <code>SortMergeJoin</code> with two exchanges           | Broadcast the small side (48.5)                 |
| Ignoring skew                                            | One task runs for hours; memory errors                  | AQE, salting, or handle the hot key separately  |
| <code>collect()</code> on a large <code>DataFrame</code> | Driver runs out of memory                               | Aggregate, sample, or write to files (48.7)     |
| Python UDFs in hot paths                                 | Slow stages; optimizer can't push filters               | Use built-in functions                          |
| Caching everything                                       | Memory pressure, slower shuffles                        | Cache only what's reused                        |
| Thousands of tiny output files                           | Later reads get slower every week                       | Aim for 128–512 MB files                        |
| Comparing tools without comparing answers                | A "fast" query that's wrong                             | Check the numbers match first (48.6)            |
| Treating one machine's benchmark as universal            | Wrong tool chosen for a bigger job                      | State the hardware; know where the crossover is |

## In the real world: the cluster that was slower than the laptop

Riverstone's plant team wanted a weekly report: for each machine, how long it ran above its safe temperature, and how that lined up with scrap. The sensor archive had reached about 300 million readings.

The contractor who took the job had done large Spark work before, and quoted a managed Spark cluster. Within three weeks there was a cluster, a notebook, and a job that took 26 minutes and cost more per month than the warehouse.

Meera asked the one question that changed the project: *"How much data does the report actually read?"*

The answer was a week at a time: about 12 million readings, filtered to one condition, grouped by machine. The cluster was reading the whole archive every run, because the files were partitioned by machine rather than by date, and the report's date filter couldn't prune anything.

Two changes followed. The archive was rewritten partitioned by date, with machine as a sorted column inside each day's files. Then the same report was written in DuckDB, reading the week's folders directly from cloud storage, and it finished in under a minute on a small virtual machine.

The cluster wasn't wrong in principle. It was solving a problem the storage layout had created. Riverstone kept Spark for one genuinely large job, the yearly rebuild across the full archive, and ran everything else on a single machine.

At the review, the contractor made the point that mattered most. "The 26 minutes weren't Spark's fault," he said. "We were reading 300 million rows to answer a question about 12 million. No engine fixes that."

#### What made this work.

- **Meera asked about the data the job touches**, not the size of the archive.
- **The storage layout was the real problem**; the engine was a symptom.
- **They kept Spark where it earned its place** instead of treating the choice as ideological.

---

## Tools

---

- **Python 3.12** with `pyspark`, `duckdb`, `polars`, `pyarrow`, and **Java 17 or 21** (Spark needs a Java runtime; check with `java -version`).
  - **The Chapter 48 companion folder** (`companion/ch48/`): `make_sensor_data.py` (generates the dataset) and `time_three_tools.py` (the timing script from section 48.6). Run Python from that folder.
  - **Versions used for the outputs shown:** Python 3.12.3, PySpark 4.2.0, DuckDB 1.5.5, Polars 1.44.2, PyArrow 25.0.1, Java 21, on a container with 1 CPU core and 3 GB of memory.
  - **Spark's web interface:** while a session is running, open `http://localhost:4040` to see stages, tasks, and the time spent in shuffles. It's the best way to see skew.
-

## The project: three questions, three engines

---

**Goal:** answer the plant team's questions on the sensor data, choose an engine with evidence, and be able to defend the choice.

**Option A: Riverstone.** Use the generated sensor dataset.

**Option B: your own data.** Any dataset of at least a few million rows that you're allowed to use, stored as Parquet.

### Steps

1. **Generate the data** and record its size on disk and its row count.
2. **Answer three questions** in DuckDB, Polars, and PySpark, and confirm all three give identical numbers: a. For each machine, the share of readings above 200 °C in December. b. For each plant and week, total units made and scrap percentage. c. The ten machine-days with the highest scrap rate, with at least 5,000 readings that day.
3. **Time each engine** on each question, three runs, and report the fastest. State your hardware.
4. **Read the Spark plan** for question (c). Write down every exchange and what causes it.
5. **Make one query faster** by pruning, broadcasting, or reducing shuffles. Show the plan before and after, and the timing change.
6. **Break something on purpose:** filter on `F.year(F.col("reading_ts"))` instead of the partition column, and show what happens to `PartitionFilters` and to the timing.
7. **Write a recommendation** of half a page: which engine for this report, at what data size you'd change your mind, and what it would cost to be wrong.

### Stretch goals

- Repeat with the dataset ten times larger (change `DAYS` in the generator) and see whether the ranking changes.
- Write the output of question (b) as Parquet partitioned by week, and measure how much faster the next read gets.
- Introduce skew by duplicating one machine's rows, and show what adaptive query execution does about it.

---

## You've got it when...

---

- ☐ You can argue, with numbers, whether a dataset needs a distributed engine.
- ☐ You can explain partitions, workers, the driver, and narrow versus wide operations.
- ☐ You can say what a shuffle is and why it dominates cost.
- ☐ You can write PySpark to read, filter, group, join, and write Parquet.
- ☐ You can explain lazy execution and name three actions.
- ☐ You can read a physical plan, find every exchange, and say what caused each one.

- ☐ You can prove partition pruning and column pruning are happening.
  - ☐ You can turn a sort-merge join into a broadcast join, and explain when not to.
  - ☐ You can describe skew, its symptoms, and three fixes.
  - ☐ You can compare engines on correctness first, then speed, and state your hardware.
  - ☐ You can list five ways a Spark job surprises people, including `collect()` and small files.
  - ☐ You can place Databricks, EMR, Trino, and Ray on the map.
- 

## Recap

---

- **Most "big data" isn't.** Ask what the job actually reads, after filtering.
  - A distributed engine splits data into **partitions**, runs work on **workers**, and coordinates from the **driver**.
  - **Narrow** operations stay inside a partition and are cheap; **wide** operations need a **shuffle** and are expensive. Every `Exchange` in a plan is a shuffle.
  - **Lazy execution** means nothing runs until an **action**; the plan is built from the whole query.
  - **Partition pruning** (folders skipped) and **column pruning** (columns not read) remove work before it starts; check `PartitionFilters` and `ReadSchema`.
  - A **broadcast join** sends a small table to every worker so the big one never moves.
  - **Skew** makes one worker do most of the work; adaptive execution, salting, or special-casing the hot key fix it.
  - On one machine, at 19.87 million rows, **DuckDB (0.38 s) and Polars (0.47 s) beat Spark (2.01 s)**; the numbers are machine-specific, and Spark's advantage appears when data outgrows one machine.
  - Avoid `collect()` on large results, thousands of tiny files, careless caching, and Python UDFs in hot paths.
  - Learn SQL and one single-machine engine deeply, understand Spark's model, and add a platform when a job needs it.
- 

## Practice exercises

---

### Warm-up

1. Classify each operation as narrow or wide: (a) `filter`; (b) `groupBy(...).agg(...)`; (c) `select`; (d) `join` on a key; (e) `withColumn` with arithmetic; (f) `orderBy`.
2. In the plan in section 48.4, name every line that causes data to move between workers, and say what it's ordering by.
3. A colleague's job on 4 million rows takes 90 seconds in Spark on their laptop and 3 seconds in DuckDB. Give two reasons, and say what you'd recommend.

## Core

4. Write the PySpark for: for each machine in December, the number of readings, the share above 200 °C as a percentage, and the total units made, ordered by that share, highest first.
5. Using the plan from section 48.5's pruning example, explain the difference between `PartitionFilters` and `PushedFilters`. Which one would disappear if the data weren't partitioned by date, and what would the query then cost?
6. The join in section 48.5 moved 19.87 million rows to match them with 3. Explain why Spark did that when broadcasting was switched off, and why `F.broadcast()` is still worth writing even when the automatic threshold would catch it.
7. `spark.sql.shuffle.partitions` is 200 by default. Explain what goes wrong with 200 partitions on a one-core machine, and what goes wrong with 8 partitions on a 200-core cluster.
8. From the timings in section 48.6, calculate how many times slower Spark was than DuckDB at each data size. What does the change between the two sizes suggest about where the crossover might be?
9. A job writes its output with `df.write.parquet(...)` after a group-by, and produces 200 files of 400 KB each. Explain the problem this creates for tomorrow's reader, and write the one line that fixes it.

## Stretch

10. A join between readings and a 2 GB table of machine settings is slow, and the plan shows `SortMergeJoin` with two exchanges. Broadcasting isn't possible. List three other things you'd try, in order, and what you'd check in the plan after each.
11. Half of Riverstone's readings come from one high-volume machine, and a join on `machine_id` has one task that never finishes. Describe the salting fix in detail, including what happens to the small side, and what you'd check afterwards.
12. Design the storage layout for the sensor archive so that the weekly report in "In the real world" prunes correctly: partition columns, file sizes, sort order inside files, and what you'd do about the existing files.

## Think about it (no code needed)

13. Spark appears in far more job descriptions than most teams' daily work needs. How would you prepare for those interviews without building a platform you don't need? Be specific about what you'd learn and what you'd build.
14. "The cluster wasn't wrong; the storage layout was." Explain to a non-technical manager why moving the same data into a bigger machine can fail to fix a slow report, using the votes-and-boxes analogy from the start of the chapter.

## Key terms

---

distributed computing · partition · worker (executor) · driver · narrow operation · wide operation · shuffle · exchange · lazy execution · action · query plan · physical plan · adaptive query execution (AQE) · partition pruning · column pruning · pushed filter · broadcast join · sort-merge join · skew · salting · cache · collect · small files problem · user-defined function (UDF) · local mode · DuckDB · Polars · PySpark · Databricks · EMR · Dataproc · Trino · Ray

*(All terms are defined in the Glossary, Appendix A.)*

---

## Where this leads

---

- **Chapter 49, Storage, Warehouses & Lakehouses**, explains the layouts this chapter depends on: partitioning, file sizes, table formats with transactions and time travel, and cloud warehouse costs.
  - **Chapter 50, Streaming & Real-Time**, uses the same sensor data as a never-ending stream, with Spark Structured Streaming.
  - **Chapter 47** applies: tests and reconciliation matter more at this scale, not less, and they must be cheap to run.
  - **Chapter 52, The Cloud, Containers & Infrastructure as Code**, covers running Spark somewhere other than your laptop, and what it costs.
  - **Chapter 56** uses distributed compute for model training and feature pipelines.
  - **Part VIII:** Spark questions appear in the data engineering interview chapters, and "how would you process X" design cases in Chapter 77.
- 

## Answers to practice exercises

---

*(In the finished book these move to Appendix G.)*

1. Narrow: (a) `filter`, (c) `select`, (e) `withColumn` with arithmetic. Wide: (b) `groupBy().agg()`, (d) `join` on a key (unless it's broadcast, in which case the big side isn't shuffled), (f) `orderBy` (a global sort needs data moved so ranges are in order).
2. One line: `Exchange hashpartitioning(machine_id, 8)`. It redistributes rows by a hash of `machine_id` into 8 partitions, so every row for a machine lands on the same worker. `ENSURE_REQUIREMENTS` means the aggregate above it needs that arrangement. The `HashAggregate ... partial_sum` below it is what keeps the shuffle small: only partial totals cross, not raw rows.
3. Two reasons: (1) **Overhead:** Spark starts a JVM, plans, and coordinates tasks; at 4 million rows that setup is most of the runtime, while DuckDB runs in the same process. (2) **Too many shuffle partitions:** with the default 200, the job creates hundreds of tiny tasks on a machine with a handful of cores.

Recommendation: keep it in DuckDB (or Polars), and revisit only if the data grows past what the machine can hold. If Spark must be used, set `spark.sql.shuffle.partitions` to a few times the core count.

#### 4. A sample answer:

```
december = readings.filter(F.col("reading_date").between("2025-12-01", "2025-12-31"))

answer = (december.groupBy("machine_id")
    .agg(F.count("*").alias("readings"),
        F.round(F.avg((F.col("temperature_c") > 200).cast("int")) * 100,
2).alias("hot_pct"),
        F.sum("units_made").alias("units"))
    .orderBy(F.col("hot_pct").desc()))
answer.show(10, truncate=False)
```

The filter is on the partition column, so only December's folders are read; `(condition).cast("int")` then averaged is the usual way to turn a share into a percentage.

5. **PartitionFilters** are applied to the *folder structure*: Spark decides which `reading_date=` folders to open at all, so unmatched files are never read. **PushedFilters** are applied *inside* the Parquet files as they're read, using each file's own row-group statistics to skip chunks, and then row by row. Without date partitioning, **PartitionFilters** would be empty; the date condition would become an ordinary pushed filter, Spark would open all 92 days of files, and the query would read roughly 92 times as much data (though Parquet statistics would still let it skip some row groups).

6. With broadcasting switched off, Spark's only general way to match rows is to put equal keys on the same worker, which means hash-partitioning **both** sides by `machine_id`: the 19.87 million-row side gets written to shuffle files, sent over the network, and read back. `F.broadcast()` is still worth writing because Spark's automatic decision rests on an **estimate** of the small side's size, and estimates go wrong after filters, joins, or aggregations, especially with no table statistics. Being explicit also documents the intent for the next reader. The caution: broadcasting a table that isn't actually small can run the driver and every worker out of memory.

7. With **200 partitions on one core**, every shuffle produces 200 small tasks that run more or less one after another, each with scheduling overhead and a tiny file; the job spends most of its time coordinating. With **8 partitions on 200 cores**, only 8 tasks exist, so at most 8 cores do anything and the other 192 sit idle, and each task handles a large slice that may not fit comfortably in memory. The rule of thumb is a few times the total core count, adjusted so each partition holds roughly 100–200 MB.

8. At 216,000 rows:  $0.73 \div 0.01 = 73$  **times slower**. At 19.87 million rows:  $2.01 \div 0.38 \approx 5.3$  **times slower**. The ratio fell sharply as the data grew, which is what you'd expect if a fixed overhead is being spread over more work. Extrapolating a ratio isn't a prediction, but the direction suggests the crossover lies well beyond this size, and on this one-core machine may never arrive: Spark's real advantage is adding machines, which this setup can't do.

9. 200 files of 400 KB means the next reader opens 200 files to read 80 MB, and each open costs a round trip, especially on cloud storage: the "small files problem". Metadata handling and task scheduling then dominate the read. The fix is to reduce the number of output partitions before

writing, for example:

```
df.coalesce(1).write.mode("overwrite").parquet("mart/weekly_scrap")
```

`coalesce(n)` merges partitions without a shuffle (cheaper); `repartition(n)` reshuffles to exactly `n` and balances them. For larger outputs, aim for files of 128–512 MB rather than a single file.

**10.** In order: (1) **Filter and select earlier**, so less of the 2 GB reaches the join, then check the plan shows the filters below the exchanges and a smaller `ReadSchema`. (2) **Pre-partition or bucket both tables on the join key** in storage, so the join can happen without a shuffle; check for the disappearance of one or both exchanges. (3) **Reduce the join itself**: aggregate the settings table to one row per key first, or join on a narrower key, then check the estimated sizes in the plan. If it's still slow, look for **skew** (one long task in the Spark UI) and apply the fixes in 48.5, and only then consider more workers.

**11.** Add a **salt** to spread the hot key. On the **big** side, add a column `salt = floor(rand() * N)` for some `N`, say 8. On the **small** side, replicate every row `N` times, once per salt value (`explode` over an array of `0...N-1`), so that whatever salt a big row got, there's a matching small row. Join on `[machine_id, salt]`. The hot machine's rows now spread across 8 partitions instead of one, so 8 workers share the work, at the cost of the small side being 8 times larger. Afterwards, check in the Spark UI that the longest task in that stage is close to the median task, and confirm row counts are unchanged, since a mistake in replicating the small side silently duplicates or drops rows.

**12.** Partition by **date** (`reading_date=YYYY-MM-DD`), because every report filters by time; that alone lets the weekly report read 7 folders out of thousands. Inside each day, **sort by machine\_id** (and write with statistics), so Parquet row-group pruning can skip most of a file when a report asks about a few machines. Aim for files of **128–512 MB** per day, which at Riverstone's volume means one file per day per plant rather than one per machine. For the existing archive, run a one-off **rewrite job**: read the old layout, write the new one partitioned by date, verify counts and totals per day match the old files (Chapter 47's reconciliation), then switch readers over and keep the old files until the next cycle has proved the new layout works. Machine-based partitioning should be dropped: it produced thousands of small files and prunes nothing for time-based reports.

**13.** A specific answer: learn Spark's **model** rather than its operations catalogue: partitions, narrow versus wide, shuffles, joins, skew, and how to read a plan. Build **one** honest project: a few tens of millions of rows in local mode, with a plan you can explain, a broadcast join before and after, and measured timings against DuckDB. Read the Spark UI for a real job. Then be ready to say, in an interview, when you would not use Spark, and why, with your own numbers. That combination (model, one real project, and judgment about when it's the wrong tool) answers most interview questions, and it's honest about what you've done. Anything beyond that is best learned on a job that actually has the data.

**14.** Using the analogy: the report is asking a question about one district's votes, but the ballots were sorted into boxes by candidate rather than by district. So whichever way you organize the counting officers, every box still has to be opened to find that district's ballots. Hiring more officers (a bigger machine or a cluster) means the opening happens in parallel, but the total work stays enormous, and coordination adds its own cost. If instead the ballots are stored in one box per district, a single officer

answers the question in minutes by opening one box. The lesson for the manager: the way the data is organized decides how much work a question needs; the size of the machine only decides how fast that work gets done.